

The drumbeat for improved software quality began in earnest as software became increasingly integrated in every facet of our lives. By the 1990s, major corporations recognized that billions of dollars each year were being wasted on software that didn’t deliver the features and functionality that were promised. Worse, both government and industry became increasingly concerned that a major software fault might cripple important infrastructure, costing tens of billions more. By the turn of the century, *CIO Magazine* trumpeted the headline, “Let’s Stop Wasting \$78 Billion a Year,” lamenting the fact that “American businesses spend billions for software that doesn’t do what it’s supposed to do” [Lev01]. Sadly, at least one survey of the state of software quality practices done in 2014 suggests that maintenance and software evolution activities make up as much as 90 percent of the total software development costs [Nan14]. Poor software quality caused by a rush to release products without adequate testing continues to plague the software industry.

KEY
CONCEPTS

cost of quality	317	quality dilemma	315
good enough	316	quality dimensions	314
liability	320	quality factors	312
machine learning	322	quantitative quality assessment	315
management actions	321	risks	319
quality	311	security	320



QUICK LOOK

What is it? The answer isn’t as easy as you might think. You know quality when you see it, and yet, it can be an elusive thing to define. But for computer software, quality is something that we must define, and that’s what we’ll do in this chapter.

Who does it? Everyone—software engineers, managers, all stakeholders—involved in the software process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

What are the steps? To achieve high-quality software, four activities must occur: proven software engineering process and practice, solid project management, comprehensive quality control, and the presence of a quality assurance infrastructure.

What is the work product? Software that meets its customer’s needs, performs accurately and reliably, and provides value to all who use it.

How do I ensure that I’ve done it right? Track quality by examining the results of all quality control activities, and measure quality by examining errors before delivery and defects released to the field.

Today, software quality remains an issue, but who is to blame? Customers blame developers, arguing that sloppy practices lead to low-quality software. Developers blame customers (and other stakeholders), arguing that irrational delivery dates and a continuing stream of changes force them to deliver software before it has been fully validated. Who's right? *Both*—and that's the problem. In this chapter, we consider software quality as a concept and examine why it's worthy of serious consideration whenever software engineering practices are applied.

15.1 WHAT IS QUALITY?

In his mystical book *Zen and the Art of Motorcycle Maintenance*, Robert Persig [Per74] commented on the thing we call *quality*:

Quality . . . you know what it is, yet you don't know what it is. But that's self-contradictory. But some things are better than others; that is, they have more quality. But when you try to say what the quality is, apart from the things that have it, it all goes poof! There's nothing to talk about . . . Obviously some things are better than others . . . but what's the betterness? . . . So round and round you go, spinning mental wheels and nowhere finding anyplace to get traction. What the hell is Quality? What is it?

Indeed—what is it?

At a somewhat more pragmatic level, David Garvin [Gar84] of the Harvard Business School suggests that “quality is a complex and multifaceted concept” that can be described from five different points of view. The *transcendental view* argues (like Persig) that quality is something you immediately recognize but cannot explicitly define. The *user view* sees quality in terms of an end user's specific goals. If a product meets those goals, it exhibits quality. The *manufacturer's view* defines quality in terms of the original specification of the product. If the product conforms to the spec, it exhibits quality. The *product view* suggests that quality can be tied to inherent characteristics (e.g., functions and features) of a product. Finally, the *value-based view* measures quality based on how much a customer is willing to pay for a product. In reality, quality encompasses all these views and more.

Quality of design refers to the characteristics that designers specify for a product. The grade of materials, tolerances, and performance specifications all contribute to the quality of design. As higher-grade materials are used, tighter tolerances and greater levels of performance are specified, and the design quality of a product increases if the product is manufactured according to specifications.

In software development, quality of design encompasses the degree to which the design meets the functions and features specified in the requirements model. *Quality of conformance* focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

But are quality of design and quality of conformance the only issues that software engineers must consider? Robert Glass [Gla98] argues that a more “intuitive” relationship is in order:

$$\begin{aligned} \text{user satisfaction} &= \text{compliant product} + \text{good quality} \\ &\quad + \text{delivery within budget and schedule} \end{aligned}$$

At the bottom line, Glass contends that quality is important, but if the user isn't satisfied, nothing else really matters. DeMarco [DeM98] reinforces this view when he states: "A product's quality is a function of how much it changes the world for the better." This view of quality contends that if a software product provides substantial benefit to its end users, they may be willing to tolerate occasional reliability or performance problems. A modern view of software quality requires attention to customer satisfaction as well as conformance to the product requirements [Max16].

15.2 SOFTWARE QUALITY

Even the most jaded software developers will agree that high-quality software is an important goal. But how do we define *software* quality? In the most general sense, software quality can be defined as: *An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.*

There is little question that the preceding definition could be modified or extended and debated endlessly. For the purposes of this book, the definition serves to emphasize three important points:

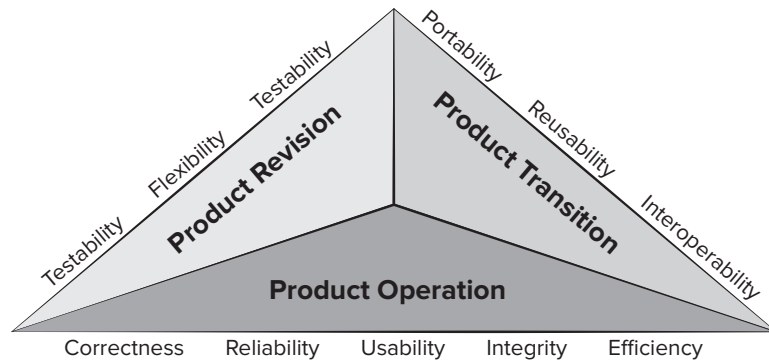
1. An *effective software process* establishes the infrastructure that supports any effort at building a high-quality software product. The management aspects of process create the checks and balances that help avoid project chaos—a key contributor to poor quality. Software engineering practices allow the developer to analyze the problem and design a solid solution—both critical to building high-quality software. Finally, umbrella activities such as change management and technical reviews have as much to do with quality as any other part of software engineering practice.
2. A *useful product* delivers the content, functions, and features that the end user desires, but as important, it delivers these assets in a reliable, error-free way. A useful product always satisfies those requirements that have been explicitly stated by stakeholders. In addition, it satisfies a set of implicit requirements (e.g., ease of use) that are expected of all high-quality software.
3. By *adding value for both the producer and user* of a software product, high-quality software provides benefits for the software organization and the end-user community. The software organization gains added value because high-quality software requires less maintenance effort, fewer bug fixes, and reduced customer support. This enables software engineers to spend more time creating new applications and less on rework. The user community gains added value because the application provides a useful capability in a way that expedites some business process. The end result is (1) greater software product revenue, (2) better profitability when an application supports a business process, and/or (3) improved availability of information that is crucial for the business.

15.2.1 Quality Factors

Several software quality models and standards have been proposed in the software engineering literature. David Garvin [Gar84] writes that quality is a multifaceted

FIGURE 15.1

McCall's
software
quality factors



phenomena and requires the use of multiple perspectives to assess it. McCall and Walters [McC77] proposed a useful way to think about and organize factors affecting software quality. Their software quality factors (shown in Figure 15.1) focus on three software product aspects: its operation characteristics, its ability to undergo change, and its adaptability to new environments. McCall's quality factors provide a basis for engineering software that provides high levels of user satisfaction by focusing on the overall user experience delivered by the software product. This cannot be done unless developers ensure that the requirements specification is correct and that defects are removed early in the software development process [Max16].

The ISO 25010 quality model is the newest standard (created in 2011 and revised in 2017).¹ This standard defines two quality models. The *quality in use* model describes five characteristics that are appropriate when considering using the product in a particular context (e.g., using the product on a specific platform by a human). The *product quality* model describes eight characteristics that focus on both the static and dynamic nature of computer systems.

- **Quality in Use Model**
 - **Effectiveness.** Accuracy and completeness with which users achieve goals
 - **Efficiency.** Resources expended to achieve user goals completely with desired accuracy
 - **Satisfaction.** Usefulness, trust, pleasure, comfort
 - **Freedom from risk.** Mitigation of economic, health, safety, and environmental risks
 - **Context coverage.** Completeness, flexibility
- **Product Quality**
 - **Functional suitability.** Complete, correct, appropriate
 - **Performance efficiency.** Timing, resource utilization, capacity
 - **Compatibility.** Coexistence, interoperability

¹ The ISO 25010 can be found at <https://www.iso.org/standard/35733.html>.

- **Usability.** Appropriateness, learnability, operability, error protection, aesthetics, accessibility
- **Reliability.** Maturity, availability, fault tolerance, recoverability
- **Security.** Confidentiality, integrity, accountability, authenticity
- **Maintainability.** Modularity, reusability, modifiability, testability
- **Portability.** Adaptability, installability, replaceability

The addition of the quality in use model helps to emphasize the importance of customer satisfaction in the assessment of software quality. The product quality model points out the importance of assessing both the functional and nonfunctional requirements for the software product [Max16].

15.2.2 Qualitative Quality Assessment

The quality dimensions and factors presented in Section 15.2.1 focus on the complete software product and can be used as a generic indication of the quality of an application. Your software team can develop a set of quality characteristics and associated questions that would probe the degree to which each factor has been satisfied.² For example, ISO 25010 identifies *usability* as an important quality factor. If you were asked to evaluate a user interface and assess its usability, how would you proceed?

Although it's tempting to develop quantitative measures for the quality factors noted in Section 15.2.1, you can also create a simple checklist of attributes that provide a solid indication that the factor is present. You might start with the sub-characteristics suggested for usability in ISO 25010: appropriateness, learnability, operability, error protection, aesthetics, and accessibility. You and your team might decide to create a user questionnaire and a set of structured tasks for users to perform. You might observe the users while they perform these tasks and have them complete the questionnaire when they finish. We will discuss usability testing in more detail in Chapter 21.

To conduct your assessment, you and your team will need to address specific, measurable (or at least, recognizable) attributes of the interface. Your tasks might be focused on answering the following questions:

- How quickly can users determine whether the software product can be used to help them complete their task or not? (appropriateness)
- How long does it take users to learn how to use the system functions needed to complete their task? (learnability)
- Is the user able to recall how to use system functions in subsequent testing sessions without having to relearn them? (learnability)
- How long does it take users to complete tasks using the system? (operability)
- Does the system try to prevent users from making errors? (error protection)
- Does the system allow users to undo operations that may have resulted in errors? (error protection)
- Do answers give favorable responses to questions about the appearance of the user interface? (aesthetics)

² These characteristics and questions would be addressed as part of a software review (Chapter 16).

- Does the interface conform to the expectations set forth by the golden rules from Chapter 12? (accessibility)
- Does the user interface conform to the accessibility checklist items required for the intended users? (accessibility)

As the interface design is developed, the software team would review the design prototype and ask the questions noted. If the answer to most of these questions is yes, it is likely that the user interface exhibits high quality. A collection of questions similar to these would be developed for each quality factor to be assessed. In the case of usability, it is always important to observe representative users interact with the system. For some other quality factors it may be important to test the software in the wild (or at least in the production environment).

15.2.3 Quantitative Quality Assessment

In the preceding subsections, we have presented a set of qualitative factors for the “measurement” of software quality. The software engineering community strives to develop precise measures for software quality and is sometimes frustrated by the subjective nature of the activity. Cavano and McCall [Cav78] discuss this situation:

Subjectivity and specialization . . . apply to determining software quality. To help solve this problem, a more precise definition of software quality is needed as well as some way to derive quantitative measurements of software quality for objective analysis . . . Since there is no such thing as absolute knowledge, one should not expect to measure software quality exactly, for every measurement is partially imperfect.

Several software design defects can be detected using software metrics. The process consists of finding code fragments that suggest the presence of things like high coupling or unnecessary levels of complexity. Internal code attributes can be described quantitatively using software metrics. Any time software metric values computed for a code fragment fall outside the range of acceptable values, it signals the existence of a quality problem that should be investigated [Max16].

In Chapter 23, we’ll present a set of software metrics that can be applied to the quantitative assessment of software quality. In all cases, the metrics represent indirect measures; that is, we never really measure *quality* but rather some manifestation of quality. The complicating factor is the precise relationship between the variable that is measured and the quality of software.

15.3 THE SOFTWARE QUALITY DILEMMA

In an interview [Ven03] published on the Web, Bertrand Meyer discusses what we call the *quality dilemma*:

If you produce a software system that has terrible quality, you lose because no one will want to buy it. If on the other hand you spend infinite time, extremely large effort, and huge sums of money to build the absolutely perfect piece of software, then it’s going to take so long to complete and it will be so expensive to produce that you’ll be out of business anyway. Either you missed the market window, or you simply exhausted all your resources. So people in industry try to get to that magical middle ground where the product is good enough not to be rejected right away, such as during evaluation, but also not the object of so much perfectionism and so much work that it would take too long or cost too much to complete.

It's fine to state that software engineers should strive to produce high-quality systems. It's even better to apply good practices in your attempt to do so. But the situation discussed by Meyer is real life and represents a dilemma for even the best software engineering organizations. When you're faced with the quality dilemma (and everyone is faced with it at one time or another), try to achieve balance—enough effort to produce acceptable quality without burying the project.

15.3.1 “Good Enough” Software

Stated bluntly, if we are to accept the argument made by Meyer, is it acceptable to produce “good enough” software? The answer to this question must be yes, because software companies do it every day [Rod17]. They create software with known bugs and deliver it to a broad population of end users. They recognize that some of the functions and features delivered in version 1.0 may not be of the highest quality and plan for improvements in version 2.0. They do this knowing that some customers will complain, but they recognize that time to market may trump better quality as long as the delivered product is “good enough.”

Exactly what is “good enough”? Good enough software delivers high-quality functions and features that end users desire, but at the same time it delivers other more obscure or specialized functions and features that contain known bugs. The software vendor hopes that the vast majority of end users will overlook the bugs because they are so happy with other application functionality.

This idea may resonate with many readers. If you're one of them, we can only ask you to consider some of the arguments against “good enough.” It is true that “good enough” may work in some application domains and for a few major software companies. After all, if a company has a large marketing budget and can convince enough people to buy version 1.0, it has succeeded in locking them in. As we noted earlier, it can argue that it will improve quality in subsequent versions. By delivering a good enough version 1.0, it has cornered the market.

If you work for a small company, be wary of this philosophy. When you deliver a good enough (buggy) product, you risk permanent damage to your company's reputation. You may never get a chance to deliver version 2.0 because bad buzz may cause your sales to plummet and your company to fold.

If you work in certain application domains (e.g., real-time embedded software) or build application software that is integrated with hardware (e.g., automotive software, telecommunications software), delivering software with known bugs can be negligent and open your company to expensive litigation. In some cases, it can even be criminal. No one wants good enough aircraft avionics software! Ebert writes that the software process model should provide clear criteria to guide developers to determine what is really “good enough” for the intended application domain [Ebe14].

So, proceed with caution if you believe that “good enough” is a shortcut that can solve your software quality problems. It can work, but only for a few and only in a limited set of application domains.³

³ A worthwhile discussion of the pros and cons of “good enough” software can be found in [Bre02].

15.3.2 The Cost of Quality

The argument goes something like this: *We know that quality is important, but it costs us time and money—too much time and money to get the level of software quality we really want.* On its face, this argument seems reasonable (see Meyer’s comments earlier in this section). There is no question that quality has a cost, but lack of quality also has a cost—not only to end users who must live with buggy software, but also to the software organization that has built and must maintain it. The real question is this: *Which cost should we be worried about?* To answer this question, you must understand both the cost of achieving quality and the cost of low-quality software.

The *cost of quality* includes all costs incurred in the pursuit of quality or in performing quality-related activities and the downstream costs of lack of quality. To understand these costs, an organization should collect metrics to provide a baseline for the current cost of quality, identify opportunities for reducing these costs, and provide a normalized basis of comparison. The cost of quality can be divided into costs associated with prevention, appraisal, and failure.

Prevention costs include (1) the cost of management activities required to plan and coordinate all quality control and quality assurance activities, (2) the cost of added technical activities to develop complete requirements and design models, (3) test planning costs, and (4) the cost of all training associated with these activities. Don’t be afraid to incur significant prevention costs. Rest assured that your investment will provide an excellent return.

Appraisal costs include activities to gain insight into product condition the “first time through” each process. Examples of appraisal costs include: (1) the cost of conducting technical reviews (Chapter 16) for software engineering work products, (2) the cost of data collection and metrics evaluation (Chapter 23), and (3) the cost of testing and debugging (Chapters 19 through 21).

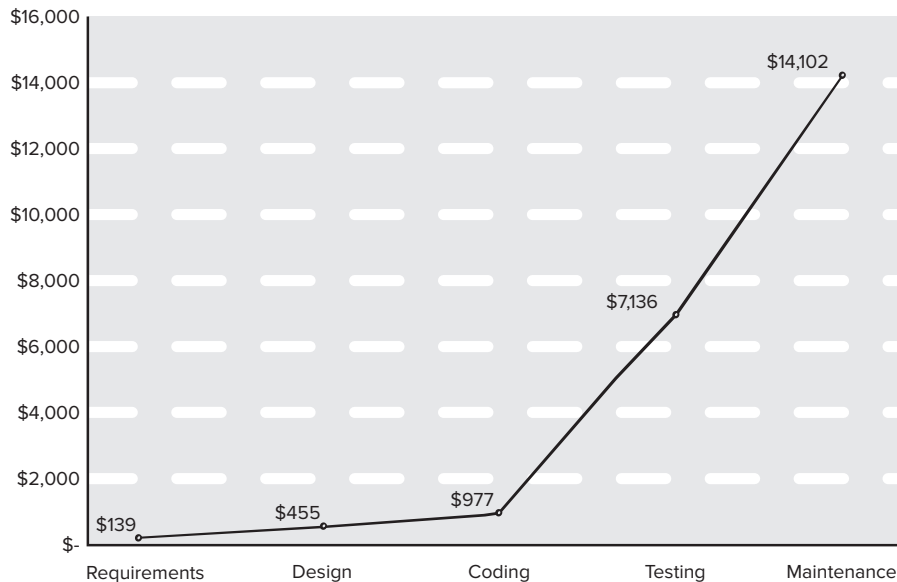
Failure costs are those that would disappear if no errors appeared before shipping a product to customers. Failure costs may be subdivided into internal failure costs and external failure costs. *Internal failure costs* are incurred when you detect an error in a product prior to shipment. Internal failure costs include: (1) the cost required to perform rework (repair) to correct an error, (2) the cost that occurs when rework inadvertently generates side effects that must be mitigated, and (3) the costs associated with the collection of quality metrics that allow an organization to assess the modes of failure. *External failure costs* are associated with defects found after the product has been shipped to the customer. Examples of external failure costs are complaint resolution, product return and replacement, help line support, and labor costs associated with warranty work. A poor reputation and the resulting loss of business is another external failure cost that is difficult to quantify but nonetheless very real. Bad things happen when low-quality software is produced.

In an indictment of software developers who refuse to consider external failure costs, Cem Kaner [Kan95] states:

Many of the external failure costs, such as goodwill, are difficult to quantify, and many companies therefore ignore them when calculating their cost-benefit tradeoffs. Other external failure costs can be reduced (e.g. by providing cheaper, lower-quality, post-sale support, or by charging customers for support) without increasing customer satisfaction. By ignoring the costs to our customers of bad products, quality engineers encourage quality-related decision-making that victimizes our customers, rather than delighting them.

FIGURE 15.2**Relative cost of correcting errors and defects**

Source: Boehm, Barry and Basili, Victor R., "Software Defect Reduction Top 10 List," IEEE Computer, vol. 34, no. 1, January 2001.



As expected, the relative costs to find and repair an error or defect increase dramatically as we go from prevention to detection to internal failure to external failure costs. Figure 15.2, based on data collected by Boehm and Basili [Boe01b] and illustrated by Cigital Inc. [Cig07], illustrates this phenomenon.

The industry average cost to correct a defect during code generation is approximately \$977 per error. The industry average cost to correct the same error if it is discovered during system testing is \$7,136 per error. Cigital Inc. [Cig07] considers a large application that has 200 errors introduced during coding.

According to industry average data, the cost of finding and correcting defects during the coding phase is \$977 per defect. Thus, the total cost for correcting the 200 “critical” defects during this phase ($200 \times \$977$) is approximately \$195,400.

Industry average data shows that the cost of finding and correcting defects during the system testing phase is \$7,136 per defect. In this case, assuming that the system testing phase revealed approximately 50 critical defects (or only 25% of those found by Cigital in the coding phase), the cost of finding and fixing those defects ($50 \times \$7,136$) would have been approximately \$356,800. This would also have resulted in 150 critical errors going undetected and uncorrected. The cost of finding and fixing these remaining 150 defects in the maintenance phase ($150 \times \$14,102$) would have been \$2,115,300. Thus, the total cost of finding and fixing the 200 defects after the coding phase would have been \$2,472,100 ($\$2,115,300 + \$356,800$).

Even if your software organization has costs that are half of the industry average (most have no idea what their costs are!), the cost savings associated with early quality control and assurance activities (conducted during requirements analysis and design) are compelling.

SAFEHOME



Quality Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: I was looking at an industry report on the costs of repairing software defects. It's pretty sobering.

Jamie: We're already working on developing test cases for each functional requirement.

Doug: That's good, but I was noticing that it costs eight times as much to repair a defect that is discovered in testing than it does if the defect is caught and repaired during coding.

Vinod: We're using pair programming, so we should be able to catch most of the defects during coding.

Doug: I think you're missing the point. Quality is more than simply removing coding errors. We need to look at the project quality goals and ensure that the evolving software products are meeting them.

Jamie: Do you mean things like usability, security, and reliability?

Doug: Yes, I do. We need to build checks into the software process to monitor our progress toward meeting our quality goals.

Vinod: Can't we finish the first prototype and then check it for quality?

Doug: I'm afraid not. We must establish a culture of quality early in the project.

Vinod: What do you want us to do, Doug?

Doug: I think we will need to find a technique that will allow us to monitor the quality of the *SafeHome* products. Let's think about this and revisit this again tomorrow.

15.3.3 Risks

In Chapter 1 of this book, we wrote, “people bet their jobs, their comforts, their safety, their entertainment, their decisions, and their very lives on computer software. It better be right.” The implication is that low-quality software increases risks for both the developer and the end user.⁴ In the preceding subsection, we discussed one of these risks (cost). But the downside of poorly designed and implemented applications does not always stop with dollars and time. An extreme example [Gag04] might serve to illustrate.

Throughout the month of November 2000 at a hospital in Panama, 28 patients received massive overdoses of gamma rays during treatment for a variety of cancers. In the months that followed, 5 of these patients died from radiation poisoning and 15 others developed serious complications. What caused this tragedy? A software package, developed by a U.S. company, was modified by hospital technicians to compute modified doses of radiation for each patient.

4 In an article titled “And the ‘Most Shocking Software Failure’ Award Goes To . . .” Chelsea Frischnecht provides a few brief examples of what can go wrong. The article can be found at <https://www.tricentis.com/blog/2017/03/01/software-fail-awards/>.

The three Panamanian medical physicists, who tweaked the software to provide additional capability, were charged with second-degree murder. The U.S. company was faced with serious litigation in two countries. Gage and McCormick comment:

This is not a cautionary tale for medical technicians, even though they can find themselves fighting to stay out of jail if they misunderstand or misuse technology. This also is not a tale of how human beings can be injured or worse by poorly designed or poorly explained software, although there are plenty of examples to make the point. This is a warning for any creator of computer programs: that software quality matters, that applications must be foolproof, and that—whether embedded in the engine of a car, a robotic arm in a factory or a healing device in a hospital—poorly deployed code can kill.

Poor quality leads to risks, some of them very serious.⁵

15.3.4 Negligence and Liability

The story is all too common. A governmental or corporate entity hires a major software developer or consulting company to analyze requirements and then design and construct a software-based “system” to support some major activity. The system might support a major corporate function (e.g., pension management) or some governmental function (e.g., health care administration or homeland security).

Work begins with the best of intentions on both sides, but by the time the system is delivered, things have gone bad. The system is late, fails to deliver desired features and functions, is error-prone, and does not meet with customer approval. Litigation ensues.

In most cases, the customer claims that the developer has been negligent (in the manner in which it has applied software practices) and is therefore not entitled to payment. The developer often claims that the customer has repeatedly changed its requirements and has subverted the development partnership in other ways. In every case, the quality of the delivered system comes into question.

15.3.5 Quality and Security

As the criticality of Web-based and mobile systems grows, application security has become increasingly important. Stated simply, software that does not exhibit high quality is easier to hack, and as a consequence, low-quality software can indirectly increase the security risk with all its attendant costs and problems.

In an interview in *ComputerWorld*, author and security expert Gary McGraw comments [Wil05]:

Software security relates entirely and completely to quality. You must think about security, reliability, availability, dependability—at the beginning, in the design, architecture, test, and coding phases, all through the software life cycle [process]. Even people aware of the software security problem have focused on late life-cycle stuff. The earlier you find the software problem, the better. And there are two kinds of software problems. One is bugs, which are implementation problems. The other is software flaws—architectural problems in the design. People pay too much attention to bugs and not enough on flaws.

To build a secure system, you must focus on quality, and that focus must begin during design. The concepts and methods discussed in Part Two of this book lead to

⁵ In early 2019, an error in the flight control software produced by a major aircraft manufacturer was directly linked to two crashes and the deaths of 346 people.

a software architecture that reduces “flaws.” A more detailed discussion of security engineering is presented in Chapter 18.

15.3.6 The Impact of Management Actions

Software quality is often influenced as much by management decisions as it is by technology decisions. Even the best software engineering practices can be subverted by poor business decisions and questionable project management actions.

In Part Four of this book we discuss project management within the context of the software process. As each project task is initiated, a project leader will make decisions that can have a significant impact on product quality.

Estimation Decisions. A software team is rarely given the luxury of providing an estimate for a project *before* delivery dates are established and an overall budget is specified. Instead, the team conducts a “sanity check” to ensure that delivery dates and milestones are rational. In many cases there is enormous time-to-market pressure that forces a team to accept unrealistic delivery dates. As a consequence, shortcuts are taken, activities that lead to higher-quality software may be skipped, and product quality suffers. If a delivery date is irrational, it is important to hold your ground. Explain why you need more time, or alternatively, suggest a subset of functionality that can be delivered (with high quality) in the time allotted.

Scheduling Decisions. When a software project schedule is established (Chapter 25), tasks are sequenced based on dependencies. For example, because component **A** depends on processing that occurs within components **B**, **C**, and **D**, component **A** cannot be scheduled for testing until components **B**, **C**, and **D** are fully tested. A project schedule would reflect this. But if time is very short, and **A** must be available for further critical testing, you might decide to test **A** without its subordinate components (which are running slightly behind schedule), so that you can make it available for other testing that must be done before delivery. After all, the deadline looms. As a consequence, **A** may have defects that are hidden, only to be discovered much later. Quality suffers.

Risk-Oriented Decisions. Risk management (Chapter 26) is one of the key attributes of a successful software project. You really do need to know what might go wrong and establish a contingency plan if it does. Too many software teams prefer blind optimism, establishing a development schedule under the assumption that nothing will go wrong. Worse, they don’t have a way of handling things that do go wrong. As a consequence, when a risk becomes a reality, chaos reigns, and as the degree of craziness rises, the level of quality invariably falls.

The software quality dilemma can best be summarized by stating Meskimen’s law: *There’s never time to do it right, but always time to do it over again.* Our advice: Taking the time to do it right is almost never the wrong decision.

15.4 ACHIEVING SOFTWARE QUALITY

Software quality doesn’t just appear. It is the result of good project management and solid software engineering practice. Management and practice are applied within the context of four broad activities that help a software team achieve high software quality: software engineering methods, project management techniques, quality control actions, and software quality assurance.

15.4.1 Software Engineering Methods

If you expect to build high-quality software, you must understand the problem to be solved. You must also be capable of creating a design that conforms to the problem while at the same time exhibiting characteristics that lead to software that exhibits the quality dimensions and factors discussed in Section 15.2.

In Part Two of this book, we presented a wide array of concepts and methods that can lead to a reasonably complete understanding of the problem and a comprehensive design that establishes a solid foundation for the construction activity. If you apply those concepts and adopt appropriate analysis and design methods, the likelihood of creating high-quality software will increase substantially.

15.4.2 Project Management Techniques

The impact of poor management decisions on software quality has been discussed in Section 15.3.6. The implications are clear: If (1) a project manager uses estimation to verify that delivery dates are achievable, (2) schedule dependencies are understood and the team resists the temptation to use shortcuts, (3) risk planning is conducted so problems do not breed chaos, software quality will be affected in a positive way.

In addition, the project plan should include explicit techniques for quality and change management. Techniques that lead to good project management practices are discussed in Part Four of this book.

15.4.3 Machine Learning and Defect Prediction

Defect prediction [Mun17] is an important part of identifying software components that may have quality concerns. Defect prediction models use statistical techniques to examine the relationships among combinations of software metrics and software components containing known software defects. They can be an efficient and effective way for software developers to quickly identify defect-prone classes. This can reduce costs and development times [Mal16].

Machine learning is an application of artificial intelligence (AI) techniques that provide systems with the ability to learn and improve from experience without being explicitly programmed. Stated another way, machine learning focuses on the development of computer programs that can access data and use the data to learn for themselves. Machine learning techniques can be used to automate the process of discovering predictive relationships between software metrics and defective components [Ort17], [Li16], [Mal16].

Machine learning systems process large data sets containing representative combinations of metrics for defective and nondefective software components. These data are used to tune classification algorithms. Once the system has built a prediction model through this type of training, it can be used for quality assessment and defect prediction on data associated with future software products. Building these types of classifiers is a big part of what modern data scientists do. More discussion on the use of data science and software engineering appears in Appendix 2 of this book.

15.4.4 Quality Control

Quality control encompasses a set of software engineering actions that help to ensure that each work product meets its quality goals. Models are reviewed to ensure that

they are complete and consistent. Code may be inspected to uncover and correct errors before testing commences. A series of testing steps is applied to uncover errors in processing logic, data manipulation, and interface communication. A combination of measurement and feedback allows a software team to tune the process when any of these work products fails to meet quality goals. Quality control activities are discussed in detail throughout the remainder of Part Three of this book.

15.4.5 Quality Assurance

Quality assurance establishes the infrastructure that supports solid software engineering methods, rational project management, and quality control actions—all pivotal if you intend to build high-quality software. In addition, quality assurance consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control actions. The goal of quality assurance is to provide management and technical staff with the data necessary to be informed about product quality, thereby gaining insight and confidence that actions to achieve product quality are working. Of course, if the data provided through quality assurance identifies problems, it is management's responsibility to address the problems and apply the necessary resources to resolve quality issues. Software quality assurance is discussed in detail in Chapter 17.

15.5 SUMMARY

Concern for the quality of the software-based systems has grown as software becomes integrated into every aspect of our daily lives. But it is difficult to develop a comprehensive description of software quality. In this chapter, quality has been defined as an effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.

A wide variety of software quality dimensions and factors has been proposed over the years. All try to define a set of characteristics that, if achieved, will lead to high software quality. McCall's and the ISO 25010 quality factors establish characteristics such as reliability, usability, maintainability, functionality, and portability as indicators that quality exists.

Every software organization is faced with the software quality dilemma. In essence, everyone wants to build high-quality systems, but the time and effort required to produce "perfect" software are simply unavailable in a market-driven world. The question becomes, Should we build software that is "good enough"? Although many companies do just that, there is a significant downside that must be considered.

Regardless of the approach that is chosen, quality does have a cost that can be discussed in terms of prevention, appraisal, and failure. Prevention costs include all software engineering actions that are designed to prevent defects in the first place. Appraisal costs are associated with those actions that assess software work products to determine their quality. Failure costs encompass the internal price of failure and the external effects that poor quality precipitates.

Software quality is achieved through the application of software engineering methods, solid management practices, and comprehensive quality control—all supported by a software quality assurance infrastructure. In the chapters that follow, quality control and assurance are discussed in some detail.

PROBLEMS AND POINTS TO PONDER

- 15.1.** Describe how you would assess the quality of a university before applying to it. What factors would be important? Which would be critical?
- 15.2.** Using the definition of software quality proposed in Section 15.2, do you think it's possible to create a useful product that provides measurable value without using an effective process? Explain your answer.
- 15.3.** Using the subattributes noted for the ISO 25010 quality factor “maintainability” in Section 15.2.1, develop a set of questions that explore whether or not these attributes are present. Follow the example shown in Section 15.2.2.
- 15.4.** Describe the software quality dilemma in your own words.
- 15.5.** What is “good enough” software? Name a specific company and specific products that you believe were developed using the good enough philosophy.
- 15.6.** Considering each of the four aspects of the cost of quality, which do you think is the most expensive and why?
- 15.7.** Do a Web search and find three other examples of “risks” to the public that can be directly traced to poor software quality. Consider beginning your search at <http://catless.ncl.ac.uk/risks>.
- 15.8.** Are *quality* and *security* the same thing? Explain.
- 15.9.** Explain why it is that many of us continue to live by Meskimen's law. What is it about the software business that causes this?

REVIEWS—A RECOMMENDED
APPROACH

16

Software reviews are a “filter” for the software process work flow. Too few, and the flow is “dirty.” Too many, and the flow slows to a trickle. Reviews are applied at various points during software engineering and serve to uncover errors and defects. Software reviews “purify” software engineering work products, including requirements and design models, code, and testing data. Using metrics, you can determine which reviews work and emphasize them and at the same time remove ineffective reviews from the flow to accelerate the process.

KEY CONCEPTS	bugs	326	errors	326
	cost effectiveness	329	informal reviews	331
	defect amplification	327	record keeping	333
	defects	326	review reporting	333
	error density	328	technical reviews	326



QUICK LOOK

What is it? You’ll make mistakes as you develop software engineering work products. There’s no shame in that—as long as you try hard, very hard, to find and correct the mistakes before they are delivered to end users. Technical reviews are the most effective mechanism for finding mistakes early in the software process.

Who does it? Software engineers perform technical reviews, also called peer reviews, with their colleagues. As we discussed in Chapters 3 and 4, sometimes it is wise to include other stakeholders in these reviews.

Why is it important? If you find an error early in the process, it is less expensive to correct. In addition, errors have a way of amplifying as the process proceeds. So a relatively minor error left untreated early in the process can be amplified into a major set of errors later in the project. Finally, reviews save time by reducing

the amount of rework that will be required late in the project.

What are the steps? Your approach to reviews will vary depending on the type of review you select. In general, six steps are employed, although not all are used for every type of review: planning, preparation, structuring the meeting, noting errors, making corrections (done outside the review), and verifying that corrections have been performed properly.

What is the work product? The output of a review is a list of issues and/or errors that have been uncovered. In addition, the technical status of the work product is also indicated.

How do I ensure that I’ve done it right? First, select the type of review that is appropriate for your development culture. Follow the guidelines that lead to successful reviews. If the reviews that you conduct lead to higher-quality software, you’ve done it right.

Many different types of reviews can be conducted as part of software engineering. Each has its place. An informal meeting around the coffee machine is a form of review, if technical problems are discussed. A formal presentation of software architecture to an audience of customers, management, and technical staff is also a form of review. In this book, however, we focus on *technical* or *peer reviews*, exemplified by *casual reviews*, *walkthroughs*, and *inspections*. A technical review (TR) is the most effective filter from a quality control standpoint. Conducted by software engineers and other stakeholders for all project team members, the TR is an effective means for uncovering errors and improving software quality.

16.1 COST IMPACT OF SOFTWARE DEFECTS

Within the context of the software process, the terms *defect* and *fault* are synonymous. Both imply a quality problem that is discovered *after* the software has been released to end users (or to another framework activity in the software process). In earlier chapters, we used the term *error* to depict a quality problem that is discovered by software engineers (or others) *before* the software is released to the end user (or to another framework activity in the software process).



Bugs, Errors, and Defects

The goal of software quality control, and in a broader sense, quality management in general, is to remove quality problems in the software. These problems are referred to by various names—*bugs*, *faults*, *errors*, or *defects*, to name a few. Are these terms synonymous, or are there subtle differences between them?

In this book we make a clear distinction between an *error* (a quality problem found *before* the software is released to other stakeholders or end users) and a *defect* (a quality problem found only *after* the software has been released to end users or other stakeholders).¹ We make this distinction because errors and defects have very different economic, business, psychological, and human impacts. As software engineers, we want to find and correct as many errors as possible before the customer and/or end user encounter them. We want to avoid

INFO

defects—because defects (justifiably) make software people look bad.

It is important to note, however, that the temporal distinction made between errors and defects in this book is *not* mainstream thinking. The general consensus within the software engineering community is that defects and errors, faults, and bugs are synonymous. That is, the point in time that the problem was encountered has no bearing on the term used to describe the problem. Part of the argument in favor of this view is that it is sometimes difficult to make a clear distinction between pre- and postrelease (e.g., consider an incremental process used in agile development).

Regardless of how you choose to interpret these terms, recognize that the point in time at which a problem is discovered does matter and that software engineers should try hard—very hard—to find problems before their customers and end users encounter them.

¹ If software process improvement is considered, a quality problem that is propagated from one process framework activity (e.g., **modeling**) to another (e.g., **construction**) can also be called a “defect” because the problem should have been found before a work product (e.g., a design model) was “released” to the next activity.

The primary objective of a formal technical review (FTR) is to find errors before they are passed on to another software engineering activity or released to the end user. The obvious benefit of technical reviews is the early discovery of errors so that they do not propagate to the next step in the software process.

A number of industry studies indicate that design activities introduce between 50 and 65 percent of all errors (and ultimately, all defects) during the software process. However, review techniques have been shown to be up to 75 percent effective [Jon86] in uncovering design flaws. By detecting and removing a large percentage of these errors, the review process substantially reduces the cost of subsequent activities in the software process. We have known this for decades, and yet there are still many developers who do not believe the time spent on reviews is almost always less than the time required to rewrite bad code [Yad17].

16.2 DEFECT AMPLIFICATION AND REMOVAL

Defect amplification is a concept originally proposed almost four decades ago [IBM81]. It helps to justify the effort expended on software reviews. In essence, defect amplification makes the following argument—an error introduced early in the software engineering work flow (e.g., during requirement modeling) and undetected, can and often will be amplified into multiple errors during design. If those errors are not uncovered (using effective reviews), they themselves may be further amplified into still more errors during coding. A single error introduced early and not uncovered and corrected can amplify into multiple errors later in the process. *Defect propagation* is a term used to describe the impact an undiscovered error has on future development activities or product behavior [Vit17].

As a development team moves deeper into the software process, the cost of finding and fixing an error grows. This simple reality is exacerbated by defect amplification and propagation because a single error may become multiple errors downstream. The cost of finding and fixing a single error can be significant, but the cost to find and fix multiple errors propagated by a single earlier error is substantially more significant.

To conduct reviews, you must expend time and effort, and your development organization must spend money. However, the reality of defect amplification and propagation leaves little doubt that you can pay now or pay much more later. This is what *technical debt* (Chapter 11) is all about [Xia16] [Vit17].

16.3 REVIEW METRICS AND THEIR USE

Technical reviews are one of many actions that are required as part of good software engineering practice. Each action requires dedicated human effort. Because available project effort is finite, it is important for a software engineering organization to understand the effectiveness of each action by defining a set of metrics (Chapter 23) that can be used to assess their efficacy.

Although many metrics can be defined for technical reviews, a relatively small subset can provide useful insight. The following review metrics can be collected for each review that is conducted:

- **Preparation effort, E_p .** The effort (in person-hours) required to review a work product prior to the actual review meeting
- **Assessment effort, E_a .** The effort (in person-hours) that is expended during the actual review
- **Rework effort, E_r .** The effort (in person-hours) that is dedicated to the correction of those errors uncovered during the review
- **Review effort, E_{review} .** Represents the sum of effort measures for reviews:

$$E_{\text{review}} = E_p + E_a + E_r$$

- **Work product size (WPS).** A measure of the size of the work product that has been reviewed (e.g., the number of UML models, the number of document pages, or the number of lines of code)
- **Minor errors found, $\text{Err}_{\text{minor}}$.** The number of errors found that can be categorized as minor (requiring less than some prespecified effort to correct)
- **Major errors found, $\text{Err}_{\text{major}}$.** The number of errors found that can be categorized as major (requiring more than some prespecified effort to correct)
- **Total errors found, Err_{tot} .** Represents the sum of the errors found:

$$\text{Err}_{\text{tot}} = \text{Err}_{\text{minor}} + \text{Err}_{\text{major}}$$

- **Error density.** Represents the errors found per unit of work product reviewed:

$$\text{Error density} = \frac{\text{Err}_{\text{tot}}}{\text{WPS}}$$

How might these metrics be used? As an example, consider a requirements model that is reviewed to uncover errors, inconsistencies, and omissions. It would be possible to compute the error density in several different ways. Assume the requirements model contains 18 UML diagrams as part of 32 overall pages of descriptive materials. The review uncovers 18 minor errors and 4 major errors. Therefore, $\text{Err}_{\text{tot}} = 22$. Error density is 1.2 errors per UML diagram, or 0.68 errors per requirements model page.

If reviews are conducted for a number of different types of work products (e.g., requirements model, design model, code, test cases), the percentage of errors uncovered for each review can be computed against the total number of errors found for all reviews. In addition, the error density for each work product can be computed.

Once data are collected for many reviews conducted across many projects, average values for error density enable you to estimate the number of errors to be found in a new document before it is reviewed. For example, if the average error density for a requirements model is 0.68 errors per page, and a new requirements model is 40 pages long, a rough estimate suggests that your software team will find around 27 errors during the review of the document. If you find only 9 errors, you've either done an extremely good job in developing the requirements model or your review approach was not thorough enough.

It is difficult to measure the cost effectiveness of any technical review in real time. A software engineering organization can assess the effectiveness of reviews and their cost benefit only after reviews have been completed, review metrics have been collected, average data have been computed, and then the downstream quality of the software is measured (via testing).

Returning to the previous example, the average error density for requirements models was determined to be 0.68 per page. The effort required to correct a minor model error (immediately after the review) has been found to require 4 person-hours. The effort required for a major requirement error has been found to be 18 person-hours. Examining the review data collected, you find that minor errors occur about six times more frequently than major errors. Therefore, you can estimate that the average effort to find and correct a requirements error during review is about 6 person-hours.

Requirements-related errors uncovered during testing require an average of 45 person-hours to find and correct (no data are available on the relative severity of the error). Using the averages noted, we get:

$$\begin{aligned}\text{Effort saved per error} &= E_{\text{testing}} - E_{\text{reviews}} \\ &= 45 - 6 = 39 \text{ person-hours/error}\end{aligned}$$

Because 22 errors were found during the review of the requirements model, a savings of about 858 person-hours of testing effort would be achieved. And that's just for requirements-related errors. Errors associated with design and code would add to the overall benefit.

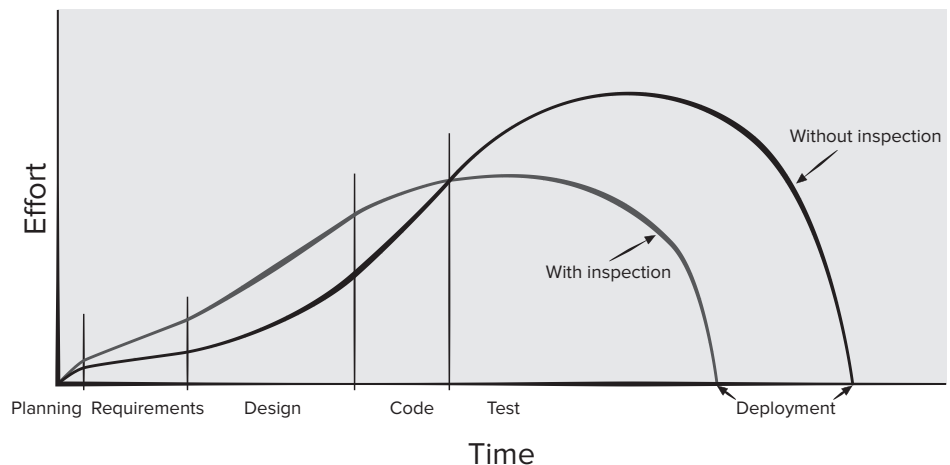
The bottom line—effort saved leads to shorter delivery cycles and improved time to market. The example presented in this section suggests this may be true. More importantly, industry data for software reviews has been collected for more than three decades and is summarized qualitatively using the graphs shown in Figure 16.1.

Referring to the figure, the effort expended when reviews are used does increase early in the development of a software increment, but this early investment for reviews pays dividends because testing and corrective effort is reduced. It is important to note

FIGURE 16.1

Effort expended with and without reviews

Source: Fagan, Michael E., "Advances in Software Inspections," IEEE Transactions on Software Engineering, vol. SE-12, no. 7, July 1986, 744–751.



that the deployment date for development with reviews is sooner than the deployment date without reviews. Reviews don't take time; they save it!

16.4 CRITERIA FOR TYPES OF REVIEWS

Technical reviews can be classified as either formal or informal or somewhere in between these two extremes. The level of formality is chosen to match the type of product to be built, the project time line, and the people who are doing the work. Figure 16.2 depicts a reference model for technical reviews [Lai02] that identifies four characteristics that contribute to the formality with which a review is conducted.

Each of the reference model characteristics helps to define the level of review formality. The formality of a review increases when (1) distinct roles are explicitly defined for the reviewers, (2) there is a sufficient amount of planning and preparation for the review, (3) a distinct structure for the review (including tasks and internal work products) is defined, and (4) follow-up by the reviewers occurs for any corrections that are made.

An element that is not presented in this model is the frequency of the reviews themselves. If you are using an agile prototyping model (Chapter 4) that contains relatively short sprints, your team may opt for less formal reviews because the reviews are happening fairly often. This usually means that defects are caught sooner and more often.

To understand the reference model, let's assume that you've decided to review the interface design for **SafeHomeAssured.com**. You can do this in a variety of different ways that range from relatively casual to extremely rigorous. If you decide that the casual approach is most appropriate, you ask a few colleagues (peers) to examine the interface prototype in an effort to uncover potential problems. All of you decide that there will be no advance preparation, but that you will evaluate the prototype in a reasonably structured way—looking at layout first, aesthetics next, navigation options after that, and so on. As the designer, you decide to take a few notes, but nothing formal.

FIGURE 16.2

Reference
model for
technical
reviews



But what if the interface is pivotal to the success of the entire project? What if human lives depended on an interface that was ergonomically sound? You might decide that a more rigorous approach was necessary. A review team would be formed. Each person on the team would have a specific role to play—leading the team, recording findings, presenting the material, and so on. Each reviewer would be given access to the work product (in this case, the interface prototype) before the review and would spend time looking for errors, inconsistencies, and omissions. A set of specific tasks would be conducted based on an agenda that was developed before the review occurred. The results of the review would be formally recorded, and the team would decide on the status of the work product based on the outcome of the review. Members of the review team might also verify that the corrections made were done properly.

In this book we consider two broad categories of technical reviews: informal reviews and more formal technical reviews. Within each category, a number of different approaches can be chosen. These are presented in the sections that follow.

16.5 INFORMAL REVIEWS

Informal reviews include a simple desk check of a software engineering work product with a colleague, a casual meeting (involving more than two people) for the purpose of reviewing a work product, or the review-oriented aspects of pair programming (Chapter 3).

A simple *desk check* or a *casual meeting* conducted with a colleague is a review. However, because there is no advance planning or preparation, no agenda or meeting structure, and no follow-up on the errors that are uncovered, the effectiveness of such reviews is considerably lower than more formal approaches. But a simple desk check can and does uncover errors that might otherwise propagate further into the software process.

One way to improve the efficacy of a desk check review is to develop a set of simple review checklists² for each major work product produced by the software team. The questions posed within the checklist are generic, but they will serve to guide the reviewers as they check the work product. For example, let's reexamine a desk check of the interface prototype for **SafeHomeAssured.com**. Rather than simply playing with the prototype at the designer's workstation, the designer and a colleague examine the prototype using a checklist for interfaces:

- Is the layout designed using standard conventions? Left to right? Top to bottom?
- Does the presentation need to be scrolled?
- Are color and placement, typeface, and size used effectively?
- Are all navigation options or functions represented at the same level of abstraction?
- Are all navigation choices clearly labeled?

2 Literally hundreds of technical review checklists can be found via a web search. For example, a useful code review checklist can be downloaded from https://courses.cs.washington.edu/courses/cse403/12wi/sections/12wi_code_review_checklist.pdf.

and so on. Any errors or issues noted by the reviewers are recorded by the designer for resolution at a later time. Desk checks may be scheduled in an ad hoc manner, or they may be mandated as part of good software engineering practice. In general, the amount of material to be reviewed is relatively small and the overall time spent on a desk check spans little more than 1 or 2 hours.

In Chapter 3, we described *pair programming* in the following manner: XP recommends that two people work together at one computer workstation to create code for a story. This provides a mechanism for real-time problem solving (two heads are often better than one) and real-time quality assurance.

Pair programming (Section 3.5.1) can be characterized as a continuous desk check. Rather than scheduling a review at some point in time, pair programming encourages continuous review as a work product (design or code) is created. The benefit is immediate discovery of errors and better work product quality.

Some software engineers argue that the inherent redundancy built into pair programming is wasteful of resources. After all, why assign two people to a job that one person can accomplish? The answer to this question can be found in Section 16.3. If the quality of the work product produced as a consequence of pair programming is significantly better than the work of an individual, the quality-related savings can more than justify the “redundancy” implied by pair programming.

16.6 FORMAL TECHNICAL REVIEWS

A *formal technical review* (FTR) is a software quality control activity performed by software engineers (and others). The objectives of an FTR are: (1) to uncover errors in function, logic, or implementation for any representation of the software; (2) to verify that the software under review meets its requirements; (3) to ensure that the software has been represented according to predefined standards; (4) to achieve software that is developed in a uniform manner; and (5) to make projects more manageable. In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and implementation. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen.

The FTR is actually a class of reviews that includes *walkthroughs* and *inspections*. Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended. In the sections that follow, guidelines similar to those for a walkthrough are presented as a representative formal technical review. If you have interest in software inspections, as well as additional information on walkthroughs, see [Rad02], [Wie02], or [Fre90].

16.6.1 The Review Meeting

Regardless of the FTR format that is chosen, every review meeting should abide by the following constraints:

- Between three and five people (typically) should be involved in the review.
- Advance preparation should occur but should require no more than 2 hours of work for each person.
- The duration of the review meeting should be less than 2 hours.

Given these constraints, it should be obvious that an FTR focuses on a specific (and small) part of the overall software. For example, rather than attempting to review an entire design, walkthroughs are conducted for each component or small group of components. By narrowing the focus, the FTR has a higher likelihood of uncovering errors.

The focus of the FTR is on a work product (e.g., a self-contained portion of a requirements model, a detailed component design, or source code for a component). The individual who has developed the work product—the *producer*—informs the project leader that the work product is complete and that a review is required. The project leader contacts a *review leader*, who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three *reviewers* for advance preparation. Each reviewer is expected to spend between 1 and 2 hours reviewing the product, making notes, and otherwise becoming familiar with the work. Concurrently, the review leader also reviews the product and establishes an agenda for the review meeting, which is typically scheduled for the next day.

The review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes on the role of a *recorder*, that is, the individual who records (in writing) all important issues raised during the review. The FTR begins with an introduction of the agenda and a brief introduction by the producer. The producer then proceeds to “walk through” the work product, explaining the material, while reviewers raise issues based on their advance preparation. When valid problems or errors are discovered, the recorder notes each. Don’t point out errors too harshly. One way to be gentle is to ask a question that enables the producer to discover the error.

At the end of the review, all attendees of the FTR must decide whether to: (1) accept the product without further modification, (2) reject the product due to severe errors (once corrected, another review must be performed), or (3) accept the product provisionally (minor errors have been encountered and must be corrected, but no additional review will be required). After the decision is made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team’s findings.

16.6.2 Review Reporting and Record Keeping

During the FTR, a reviewer (the recorder) actively records all issues that have been raised. These are summarized at the end of the review meeting, and a *review issues list* is produced. In addition, a *formal technical review summary report* is completed. A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

The review summary report is a single-page form (with possible attachments). It becomes part of the project historical record and may be distributed to the project leader and other interested parties.

The review issues list serves two purposes: (1) to identify problem areas within the product and (2) to serve as an action item checklist that guides the producer as corrections are made. An issues list is normally attached to the summary report.

You should establish a follow-up procedure to ensure that items on the issues list have been properly corrected. Unless this is done, it is possible that issues raised can “fall between the cracks.” One approach is to assign the responsibility for follow-up to the review leader.

16.6.3 Review Guidelines

Guidelines for conducting formal technical reviews must be established in advance, distributed to all reviewers, agreed upon, and then followed. A review that is uncontrolled can often be worse than no review at all. The following represents a minimum set of guidelines for formal technical reviews:

1. **Review the product, not the producer.** An FTR involves people and egos. Conducted properly, the FTR should leave all participants with a warm feeling of accomplishment. Conducted improperly, the FTR can take on the aura of an inquisition. Errors should be pointed out gently; the tone of the meeting should be loose and constructive; the intent should not be to embarrass or belittle. The review leader should conduct the review meeting to ensure that the proper tone and attitude are maintained and should immediately halt a review that has gotten out of control.
2. **Set an agenda and maintain it.** One of the key maladies of meetings of all types is drift. An FTR must be kept on track and on schedule. The review leader is chartered with the responsibility for maintaining the meeting schedule and should not be afraid to nudge people when drift sets in.
3. **Limit debate and rebuttal.** When an issue is raised by a reviewer, there may not be universal agreement on its impact. Rather than spending time debating the question, the issue should be recorded for further discussion off-line.
4. **Enunciate problem areas, but don't attempt to solve every problem noted.** A review is not a problem-solving session. The solution of a problem can often be accomplished by the producer alone or with the help of only one other individual. Problem solving should be postponed until after the review meeting.
5. **Take written notes.** It is sometimes a good idea for the recorder to make notes on a wall board, so that wording and priorities can be assessed by other reviewers as information is recorded.
6. **Limit the number of participants and insist upon advance preparation.** Two heads are better than one, but 14 are not necessarily better than 4. Keep the number of people involved to the necessary minimum. However, all review team members must prepare in advance.
7. **Develop a checklist for each product that is likely to be reviewed.** A checklist helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues. Checklists should be developed for analysis, design, code, and even testing work products.
8. **Allocate resources and schedule time for FTRs.** For reviews to be effective, they should be scheduled as a task during the software process. In addition, time should be scheduled for the inevitable modifications that will occur as the result of an FTR.

9. **Conduct meaningful training for all reviewers.** The training should stress both process-related issues and the human psychological side of reviews.
10. **Review your early reviews.** Debriefing can be beneficial in uncovering problems with the review process itself. The very first product to be reviewed should be the review guidelines themselves.

Because many variables (e.g., number of participants, type of work products, timing and length, specific review approach) have an impact on a successful review, a software organization should experiment to determine what approach works best in a local context.

In an ideal setting, every software-engineering work product would undergo a formal technical review. In the real world of software projects, resources are limited and time is short. As a consequence, reviews are often skipped, even though their value as a quality control mechanism is recognized. However, full FTR resources should be used on those work products that are likely to be error prone.

SAFEHOME



Quality Issues

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: I know we didn't spend time developing a quality plan for this project, but we're already into it and we have to consider quality . . . right?

Jamie: Sure. We've already decided that as we develop the requirements model [Chapter 8], Ed has committed to develop a testing procedure for each requirement.

Doug: That's really good, but we're not going to wait until testing to evaluate quality, are we?

Vinod: No! Of course not. We've got reviews scheduled into the project plan for this software increment. We'll begin quality control with the reviews.

Jamie: I'm a bit concerned that we won't have enough time to conduct all the reviews. In fact, I know we won't.

Doug: Hmmm. So what do you propose?

Jamie: I say we select those elements of the requirements and design model that are most critical to *SafeHome* and review them.

Vinod: But what if we miss something in a part of the model we don't review?

Jamie: Maybe . . . but I'm not sure we even have time to review every element of the models.

Vinod: What do you want us to do, Doug?

Doug: Let's steal something from Extreme Programming [Chapter 3]. We'll develop the elements of each model in pairs—two people—and conduct an informal review of each as we go. We'll then target "critical" elements for a more formal team review, but keep those reviews to a minimum. That way, everything gets looked at by more than one set of eyes, but we still maintain our delivery dates.

Jamie: That means we're going to have to revise the schedule.

Doug: So be it. Quality trumps schedule on this project.

16.7 POSTMORTEM EVALUATIONS

Many lessons can be learned if a software team takes the time to evaluate the results of a software project after the software has been delivered to end users. Baaz and his colleagues [Baa10] suggest the use of a *postmortem evaluation* (PME) as a mechanism to determine what went right and what went wrong when software engineering process and practice is applied in a specific project.

Unlike an FTR that focuses on a specific work product, it is more like a Scrum retrospective (Section 3.4.5). A PME examines the entire software project, focusing on both “*excellences* (that is, achievements and positive experiences) and *challenges* (problems and negative experiences)” [Baa10]. Often conducted in a workshop format, a PME is attended by members of the software team and stakeholders. The intent is to identify excellences and challenges and to extract lessons learned from both. The objective is to suggest improvements to both process and practice going forward. Many software engineers regard PME documents as some of the most valuable documents to save in the project archive.

16.8 AGILE REVIEWS

It’s not surprising that some software engineers balk at the thought of including reviews of any kind in agile development processes. Yet failing to catch defects early can be costly in terms of time and resources. Ignoring technical debt does not make it go away. Agile developers have the same need to find defects early (and often) that all software developers have. Admittedly, it may be somewhat more difficult to get agile developers to make use of metrics, but many of them can be collected unobtrusively.

If we take a closer look at the Scrum framework (Section 3.4), there are several places where informal and formal reviews take place. During the sprint planning meeting, user stories are reviewed and ordered according to priority, before selecting the user stories to include in the next sprint. The daily Scrum meeting is an informal way to ensure that the team members are all working on the same priorities and to catch any defects that may prevent completing the sprint on time. Agile developers often use pair programming, another informal review technique. The sprint review meeting is often conducted using guidelines similar to those discussed for a formal technical review. The code producers walk through the user stories selected for the sprint and demonstrate to the product owner that all functionality is present. Unlike the FTR, the product owner has the final word on whether to accept the sprint prototype or not.

If we look at the evaluate prototype portion of our recommended process model (Section 4.5), this task is also likely to be conducted as a formal technical review with development risk assessment added to it. We mentioned previously (Section 16.7) that the sprint retrospective meeting is really similar to the project postmortem meeting in that the development team is trying to capture its lessons learned. A major aspect of software quality assurance is being able to repeat your successes and avoid repeating your mistakes.

16.9 SUMMARY

The intent of every technical review is to find errors and uncover issues that would have a negative impact on the software to be deployed. The sooner an error is uncovered and corrected, the less likely that error will propagate to other software engineering work products and amplify itself, resulting in significantly more effort to correct it.

To determine whether quality control activities are working, a set of metrics should be collected. Review metrics focus on the effort required to conduct the review and the types and severity of errors uncovered during the review. Once metrics data are collected, they can be used to assess the efficacy of the reviews you do conduct. Industry data indicate that reviews provide a significant return on investment.

A reference model for review formality identifies the roles people play, planning and preparation, meeting structure, correction approach, and verification as the characteristics that indicate the degree of formality with which a review is conducted. Informal reviews are casual in nature but can still be used effectively to uncover errors. Formal reviews are more structured and have the highest probability of leading to high-quality software.

Informal reviews are characterized by minimal planning and preparation and little record keeping. Desk checks and pair programming fall into the informal review category.

A formal technical review is a stylized meeting that has been shown to be extremely effective in uncovering errors. Formal technical reviews establish defined roles for each reviewer, encourage planning and advance preparation, require the application of defined review guidelines, and mandate record keeping and status reporting.

PROBLEMS AND POINTS TO PONDER

16.1. Explain the difference between an *error* and a *defect*.

16.2. Why can't we just wait until testing to find and correct all software errors?

16.3. Assume that 10 errors have been introduced in the requirements model and that each error will be amplified by a factor of 2:1 into design and an additional 20 design errors are introduced and then amplified 1.5:1 into code where an additional 30 errors are introduced. Assume further that all unit testing will find 30 percent of all errors, integration will find 30 percent of the remaining errors, and validation tests will find 50 percent of the remaining errors. No reviews are conducted. How many errors will be released to the end users?

16.4. Reconsider the situation described in Problem 16.3, but now assume that requirements, design, and code reviews are conducted and are 60 percent effective in uncovering all errors at that step. How many errors will be released to the field?

16.5. Reconsider the situation described in Problems 16.3 and 16.4. If each of the errors released to the field costs \$4800 to find and correct and each error found in review costs \$240 to find and correct, how much money is saved by conducting reviews?

16.6. Describe the meaning of Figure 16.1 in your own words.

16.7. Can you think of a few instances in which a desk check might create problems rather than provide benefits?

16.8. A formal technical review is effective only if everyone has prepared in advance. How do you recognize a review participant who has not prepared? What do you do if you're the review leader?

16.9. How is technical debt addressed in agile process models?

16.10. Consider the review guidelines presented in Section 16.6.3. Which do you think is most important and why?

SOFTWARE QUALITY
ASSURANCE

17

The software engineering approach described in this book works toward a single goal: to produce on-time, high-quality software. Yet many readers will be challenged by the question: “What is software quality?”

KEY CONCEPTS	Bayesian inference	351	Six Sigma	349
	elements of software quality assurance	341	software reliability	350
	formal approaches	347	software safety	352
	genetic algorithms	352	SQA plan	354
	goals	346	SQA tasks	343
	ISO 9001:2015 standard	354	statistical software quality assurance	347



QUICK LOOK

What is it? It’s not enough to talk the talk by saying that software quality is important. You have to (1) explicitly define what is meant when you say “software quality,” (2) create a set of activities that will help ensure that every software engineering work product exhibits high quality, (3) perform quality control and assurance activities on every software project, (4) use metrics to develop strategies for improving your software process and, as a consequence, the quality of the end product.

Who does it? Everyone involved in the software engineering process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more improved, improved time to market.

What are the steps? Before software quality assurance (SQA) activities can be initiated, it is

important to define *software quality* at several different levels of abstraction. Once you understand what quality is, a software team must identify a set of SQA activities that will filter errors out of work products before they are passed on.

What is the work product? A Software Quality Assurance Plan is created to define a software team’s SQA strategy. During modeling and coding, the primary SQA work product is the output of technical reviews (Chapter 16). During testing (Chapters 19 through 21), test plans and procedures are produced. Other work products associated with process improvement may also be generated.

How do I ensure that I’ve done it right? Find errors before they become defects! That is, work to improve your defect removal efficiency (Chapter 23), thereby reducing the amount of rework that your software team must perform.

Philip Crosby [Cro79], in his landmark book on quality, provides a wry answer to this question:

The problem of quality management is not what people don't know about it. The problem is what they think they do know . . .

In this regard, quality has much in common with sex. Everybody is for it. (Under certain conditions, of course.) Everyone feels they understand it. (Even though they wouldn't want to explain it.) Everyone thinks execution is only a matter of following natural inclinations. (After all, we do get along somehow.) And, of course, most people feel that problems in these areas are caused by other people. (If only they would take the time to do things right.)

Indeed, quality is a challenging concept—one that we addressed in some detail in Chapter 15.¹

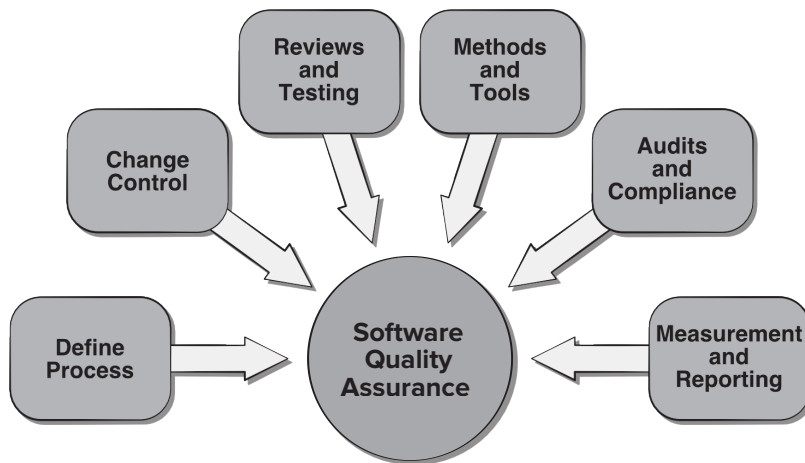
Some software developers continue to believe that software quality is something you begin to worry about after code has been generated. Nothing could be further from the truth! *Software quality assurance* (often called *quality management*) is an umbrella activity (Chapter 2) that is applied throughout the software process.

Software quality assurance (SQA) encompasses (Figure 17.1): (1) an SQA process, (2) specific quality assurance and quality control tasks (including technical reviews and a multitiered testing strategy), (3) effective software engineering practice (methods and tools), (4) control of all software work products and the changes made to them (Chapter 22), (5) a procedure to ensure compliance with software development standards (when applicable), and (6) measurement and reporting mechanisms.

In this chapter, we focus on the management issues and the process-specific activities that enable a software organization to ensure that it does “the right things at the right time in the right way.”

FIGURE 17.1

Software
quality
assurance



¹ If you have not read Chapter 15, you should do so now.

17.1 BACKGROUND ISSUES

Quality control and assurance are essential activities for any business that produces products to be used by others. Prior to the twentieth century, quality control was the sole responsibility of the craftsman who built a product. As time passed and mass production techniques became commonplace, quality control became an activity performed by people other than the ones who built the product.

The first formal quality assurance and control function was introduced at Bell Labs in 1916 and spread rapidly throughout the manufacturing world. During the 1940s, more formal approaches to quality control were suggested. These relied on measurement and continuous process improvement [Dem86] as key elements of quality management.

The history of quality assurance in software development parallels the history of quality in hardware manufacturing. During the early days of computing (1950s and 1960s), quality was the sole responsibility of the programmer. Standards for quality assurance for software were introduced in military contract software development during the 1970s and have spread rapidly into software development in the commercial world [IEE17]. Extending the definition presented earlier, software quality assurance is a “planned and systematic pattern of actions” [Sch01] that are required to ensure high quality in software. The scope of quality assurance responsibility might best be characterized by paraphrasing a once-popular automobile commercial: “Quality Is Job #1.” The implication for software is that many different constituencies have software quality assurance responsibility—software engineers, project managers, customers, salespeople, and the individuals who serve within an SQA group.

The SQA group serves as the customer’s in-house representative. That is, the people who perform SQA must look at the software from the customer’s point of view. Does the software adequately meet the quality factors noted in Chapter 15? Have software engineering practices been conducted according to preestablished standards? Have technical disciplines properly performed their roles as part of the SQA activity? The SQA group attempts to answer these and other questions to ensure that software quality is maintained.

17.2 ELEMENTS OF SOFTWARE QUALITY ASSURANCE

Software quality assurance encompasses a broad range of concerns and activities that focus on the management of software quality. These can be summarized in the following manner [Hor03]:

Standards. The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. Standards may be adopted voluntarily by a software engineering organization or imposed by the customer or other stakeholders. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.

Reviews and audits. Technical reviews are a quality control activity performed by software engineers for software engineers (Chapter 16). Their

intent is to uncover errors. Audits are a type of review performed by SQA personnel with the intent of ensuring that quality guidelines are being followed for software engineering work. For example, an audit of the review process might be conducted to ensure that reviews are being performed in a manner that will lead to the highest likelihood of uncovering errors.

Testing. Software testing (Chapters 19 through 21) is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted so that it has the highest likelihood of achieving its primary goal.

Error/defect collection and analysis. The only way to improve is to measure how you're doing. SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.

Change management. Change is one of the most disruptive aspects of any software project. If it is not properly managed, change can lead to confusion, and confusion almost always leads to poor quality. SQA ensures that adequate change management practices (Chapter 22) have been instituted.

Education. Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement (Chapter 28) and is a key proponent and sponsor of educational programs.

Vendor management. Three categories of software are acquired from external software vendors—*shrink-wrapped packages* (e.g., Microsoft Office), a *tailored shell* [Hor03] that provides a basic skeletal structure that is custom tailored to the needs of a purchaser, and *contracted software* that is custom designed and constructed from specifications provided by the customer organization. The job of the SQA organization is to ensure that high-quality software results by suggesting specific quality practices that the vendor should follow (when possible) and incorporating quality mandates as part of any contract with an external vendor.

Security management. With the increase in cyber crime and new government regulations regarding privacy, every software organization should institute policies that protect data at all levels, establish firewall protection for mobile apps, and ensure that software has not been tampered with internally. SQA ensures that appropriate process and technology are used to achieve software security (Chapter 18).

Safety. Because software is almost always a pivotal component of human-rated systems (e.g., automotive or aircraft applications), the impact of hidden defects can be catastrophic. SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.

Risk management. Although the analysis and mitigation of risk (Chapter 26) is the concern of software engineers, the SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

In addition to each of these concerns and activities, SQA works to ensure that software support activities (e.g., maintenance, help lines, documentation, and manuals) are conducted or produced with quality as a dominant concern.

17.3 SQA PROCESSES AND PRODUCT CHARACTERISTICS

As we begin a discussion of software quality assurance, it's important to note that SQA procedures and approaches that work in one software environment may not work as well in another. Even within a company that adopts a consistent approach² to software engineering, different software products may exhibit different levels of quality [Par11].

The solution to this dilemma is to understand the specific quality requirements for a software product and then select the process and specific SQA actions and tasks that will be used to meet those requirements. The Software Engineering Institute's CMMI and ISO 9000 standards are the most commonly used process frameworks. Each proposes "a syntax and semantics" [Par11] that will lead to the implementation of software engineering practices that improve product quality. Rather than instantiating either framework in its entirety, a software organization can "harmonize" the two models by selecting elements of both frameworks and matching them to the quality requirements of an individual product.

17.4 SQA TASKS, GOALS, AND METRICS

Software quality assurance is composed of a variety of tasks associated with two different constituencies—the software engineers who do technical work and an SQA group that has responsibility for quality assurance planning, oversight, record keeping, analysis, and reporting.

Modern software quality assurance is often data driven, as shown in Figure 17.2. The product stakeholders define goals and quality measures, problem areas are identified, indicators are measured, and a determination is made as to whether or not process changes are needed. Software engineers address quality (and perform quality control activities) by applying solid technical methods and measures, conducting technical reviews, and performing well-planned software testing.

17.4.1 SQA Tasks

The charter of the SQA group is to assist the software team in achieving a high-quality end product. The Software Engineering Institute recommends a set of SQA activities that address quality assurance planning, oversight, record keeping, analysis, and reporting. These activities are performed (or facilitated) by an independent SQA group that:

Prepares an SQA plan for a project. The plan is developed as part of project planning and is reviewed by all stakeholders. Quality assurance activities performed by the software engineering team and the SQA group are governed by the plan. The plan identifies evaluations to be performed, audits and

² For example, CMMI-defined process and practices (Chapter 28).

FIGURE 17.2

Software
quality
assurance



reviews to be conducted, standards that are applicable to the project, procedures for error reporting and tracking, work products that are produced by the SQA group, and feedback that will be provided to the software team.

Participates in the development of the project's software process description. The software team selects a process for the work to be performed. The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

Reviews software engineering activities to verify compliance with the defined software process. The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

Audits designated software work products to verify compliance with those defined as part of the software process. The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.

Ensures that deviations in software work and work products are documented and handled according to a documented procedure. Deviations may be encountered in the project plan, process description, applicable standards, or software engineering work products.

Records any noncompliance, and reports to senior management. Noncompliance items are tracked until they are resolved.

In addition to these activities, the SQA group coordinates the control and management of change (Chapter 22) and helps to collect and analyze software metrics.

SAFEHOME



Software Quality Assurance

The scene: Doug Miller's office as the *SafeHome* software project begins.

The players: Doug Miller, manager of the *SafeHome* software engineering team, and other members of the product software engineering team.

The conversation:

Doug: How are things going with the informal reviews?

Jamie: We're conducting informal reviews of the critical project elements in pairs as we code but before testing. It's going faster than I thought.

Doug: That's good, but I want to have Bridget Thorton's SQA group conduct audits of our work products to ensure that we're following our processes and meeting our quality goals.

Venod: Aren't they already doing the bulk of the testing?

Doug: Yes, they are. But QA is more than testing. We need to be sure that our documents are evolving along with our code and that

we're making sure we don't introduce errors as we integrate new components.

Jamie: I really don't want to be evaluated based on their findings.

Doug: No worries. The audits are focused on conformance of our work products to the requirements and the process activities we've defined. We'll only be using audit results to try to improve our processes as well as our software products.

Vinod: I must believe it's going to take more of our time.

Doug: In the long run it will save us time when we find defects earlier. It also costs less to fix defects if they're caught early.

Jamie: That sounds like a good thing then.

Doug: It's also important to identify the activities where defects were introduced and add review tasks to catch them in the future.

Vinod: That'll help us determine if we're sampling carefully enough with our review activities.

Doug: I think SQA activities will make us a better team in the long run.

17.4.2 Goals, Attributes, and Metrics

The SQA activities described in the preceding section are performed to achieve a set of pragmatic goals:

Requirements quality. The correctness, completeness, and consistency of the requirements model will have a strong influence on the quality of all work products that follow. SQA must ensure that the software team has properly reviewed the requirements model to achieve a high level of quality.

Design quality. Every element of the design model should be assessed by the software team to ensure that it exhibits high quality and that the design itself conforms to requirements. SQA looks for attributes of the design that are indicators of quality.

Code quality. Source code and related work products (e.g., other descriptive information) must conform to local coding standards and exhibit characteristics that will facilitate maintainability. SQA should isolate those attributes that allow a reasonable analysis of the quality of code.

Quality control effectiveness. A software team should apply limited resources in a way that has the highest likelihood of achieving a high-quality result. SQA analyzes the allocation of resources for reviews and testing to assess whether they are being allocated in the most effective manner.

Table 17.1 (adapted from [Hya96]) identifies the attributes that are indicators for the existence of quality for each of the goals discussed. Metrics that can be used to indicate the relative strength of an attribute are also shown.

TABLE 17.1			
Software quality goals, attributes, and metrics Source: Adapted from Hyatt, L., and Rosenberg, L., “A Software Quality Model and Metrics for Identifying Project Risks and Assessing Software Quality,” NASA SATC, 1996.	Goal	Attribute	Metric
	Requirement quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
		Completeness	Number of TBA, TBD
		Understandability	Number of sections/subsections
		Volatility	Number of changes per requirement
			Time (by activity) when change is requested
		Traceability	Number of requirements not traceable to design/code
		Model clarity	Number of UML models
	Design quality		Number of descriptive pages per model
			Number of UML errors
		Architectural integrity	Existence of architectural model
		Component completeness	Number of components that trace to architectural model
			Complexity of procedural design
		Interface complexity	Average number of pick to get to a typical function or content
			Layout appropriateness
	Code quality	Patterns	Number of patterns used
		Complexity	Cyclomatic complexity
		Maintainability	Design factors
		Understandability	Percent internal comments
			Variable naming conventions
			Percent reused components
		Reusability	Percent reused component
	QC effectiveness	Documentation	Readability index
		Resource allocation	Staff hour percentage per activity
		Completion rate	Actual vs. budgeted completion time
		Review effectiveness	See review metrics
		Testing effectiveness	Number of errors found and criticality
			Effort required to correct an error
			Origin of error

17.5 FORMAL APPROACHES TO SQA

In the preceding sections, we have argued that software quality is everyone's job and that it can be achieved through competent software engineering practice as well as through the application of technical reviews, a multitiered testing strategy, better control of software work products and the changes made to them, and the application of accepted software engineering standards and process frameworks. In addition, quality can be defined in terms of a broad array of quality attributes and measured (indirectly) using a variety of indices and metrics.

Over the past three decades, a small, but vocal, segment of the software engineering community has argued that a more formal approach to software quality assurance is required. It can be argued that a computer program is a mathematical object. A rigorous syntax and semantics can be defined for every programming language, and a rigorous approach to the specification of software requirements is available. If the requirements model (specification) and the programming language can be represented in a rigorous manner, it should be possible to apply mathematic proof of correctness to demonstrate that a program conforms exactly to its specifications.

Attempts to prove programs correct are not new. Dijkstra [Dij76a] and Linger, Mills, and Witt [Lin79], among others, advocated proofs of program correctness and tied these to the use of structured programming concepts. Although formal methods are interesting to some software engineering researchers, commercial developers rarely make use of formal methods in 2018.

17.6 STATISTICAL SOFTWARE QUALITY ASSURANCE

Statistical quality assurance reflects a growing trend throughout the industry to become more quantitative about quality. For software, statistical quality assurance implies the following steps:

1. Information about software errors and defects is collected and categorized.
2. An attempt is made to trace each error and defect to its underlying cause (e.g., nonconformance to specifications, design error, violation of standards, poor communication with the customer).
3. Using the Pareto principle (80 percent of the defects can be traced to 20 percent of all possible causes), isolate the 20 percent (the *vital few*).
4. Once the vital few causes have been identified, move to correct the problems that have caused the errors and defects.

This relatively simple concept represents an important step toward the creation of an adaptive software process in which changes are made to improve those elements of the process that introduce error.

17.6.1 A Generic Example

To illustrate the use of statistical methods for software engineering work, assume that a software engineering organization collects information on errors and defects for a period of 1 year. Some of the errors are uncovered as software is being developed. Other defects are encountered after the software has been released to its end users.

Although hundreds of different problems are uncovered, all can be tracked to one (or more) of the following causes:

- Incomplete or erroneous specifications (IES)
- Misinterpretation of customer communication (MCC)
- Intentional deviation from specifications (IDS)
- Violation of programming standards (VPS)
- Error in data representation (EDR)
- Inconsistent component interface (ICI)
- Error in design logic (EDL)
- Incomplete or erroneous testing (IET)
- Inaccurate or incomplete documentation (IID)
- Error in programming language translation of design (PLT)
- Ambiguous or inconsistent human/computer interface (HCI)
- Miscellaneous (MIS)

To apply statistical SQA, a table is built (see Table 17.2). The table indicates that IES, MCC, and EDR are the vital few causes that account for 53 percent of all errors. It should be noted, however, that IES, EDR, PLT, and EDL would be selected as the vital few causes if only serious errors are considered. Once the vital few causes are determined, the software engineering organization can begin corrective action. For example, to correct MCC, you might implement requirements gathering techniques (Chapter 7) to improve the quality of customer communication and specifications. To improve EDR, you might acquire tools for data modeling and perform more stringent data design reviews.

TABLE 17.2

Data
collection for
statistical
SQA

Error	Total		Serious		Moderate		Minor	
	No.	%	No.	%	No.	%	No.	%
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
Totals	942	100%	128	100%	379	100%	435	100%

It is important to note that corrective action focuses primarily on the vital few. As the vital few causes are corrected, new candidates pop to the top of the stack.

Statistical quality assurance techniques for software have been shown to provide substantial quality improvement (e.g., [Rya11], [Art97]). In some cases, software organizations have achieved a 50 percent reduction per year in defects after applying these techniques.

The application of the statistical SQA and the Pareto principle can be summarized in a single sentence: *Spend your time focusing on things that really matter, but first be sure that you understand what really matters!*

17.6.2 Six Sigma for Software Engineering

Six Sigma is the most widely used strategy for statistical quality assurance in industry today. Originally popularized by Motorola in the 1980s, the Six Sigma strategy “is a business-management strategy designed to improve the quality of process outputs by minimizing variation and causes of defects in processes. It is a subset of the Total Quality Management (TQM) methodology with a heavy focus on statistical applications used to reduce costs and improve quality” [Voe18]. The term *Six Sigma* is derived from six standard deviations—3.4 instances (defects) per million occurrences—implying an extremely high-quality standard. The Six Sigma methodology defines three core steps:

- *Define* customer requirements and deliverables and project goals via well-defined methods of customer communication.
- *Measure* the existing process and its output to determine current quality performance (collect defect metrics).
- *Analyze* defect metrics and determine the vital few causes.

If an existing software process is in place, but improvement is required, Six Sigma suggests two additional steps:

- *Improve* the process by eliminating the root causes of defects.
- *Control* the process to ensure that future work does not reintroduce the causes of defects.

These core and additional steps are sometimes referred to as the DMAIC (define, measure, analyze, improve, and control) method.

If an organization is developing a software process (rather than improving an existing process), the core steps are augmented as follows:

- *Design* the process to (1) avoid the root causes of defects and (2) to meet customer requirements.
- *Verify* that the process model will, in fact, avoid defects and meet customer requirements.

This variation is sometimes called the DMADV (define, measure, analyze, design, and verify) method.

A comprehensive discussion of Six Sigma is best left to resources dedicated to the subject. If you have further interest, see [Voe18], [Pyz14], and [Sne18].

17.7 SOFTWARE RELIABILITY

There is no doubt that the reliability of a computer program is an important element of its overall quality. If a program repeatedly and frequently fails to perform, it matters little whether other software quality factors are acceptable.

Software reliability, unlike many other quality factors, can be measured directly and estimated using historical and developmental data. *Software reliability* is defined in statistical terms as “the probability of failure-free operation of a computer program in a specified environment for a specified time” [Mus87]. To illustrate, program *X* is estimated to have a reliability of 0.999 over eight elapsed processing hours. In other words, if program *X* were to be executed 1000 times and require a total of 8 hours of elapsed processing time (execution time), it is likely to operate correctly (without failure) 999 times.

Whenever software reliability is discussed, a pivotal question arises: What is meant by the term *failure*? In the context of any discussion of software quality and reliability, failure is nonconformance to software requirements. Yet, even within this definition, there are gradations. Failures can be only annoying or catastrophic. One failure can be corrected within seconds, while another requires weeks or even months to correct. Complicating the issue even further, the correction of one failure may in fact result in the introduction of other errors that ultimately result in other failures.

17.7.1 Measures of Reliability and Availability

Early work in software reliability attempted to extrapolate the mathematics of hardware reliability theory to the prediction of software reliability. Most hardware-related reliability models are predicated on failure due to wear rather than failure due to design defects. In hardware, failures due to physical wear (e.g., the effects of temperature, corrosion, shock) are more likely than a design-related failure. Unfortunately, the opposite is true for software. In fact, all software failures can be traced to design or implementation problems; wear (see Chapter 1) is not a factor.

There has been an ongoing debate over the relationship between key concepts in hardware reliability and their applicability to software. Although an irrefutable link has yet to be established, it is worthwhile to consider a few simple concepts that apply to both system elements.

If we consider a computer-based system, a simple measure of reliability is *mean time between failure* (MTBF):³

$$\text{MTBF} = \text{MTTF} + \text{MTTR}$$

where the acronyms MTTF and MTTR are *mean time to failure* and *mean time to repair*,⁴ respectively.

³ It is important to note that MTBF and related measures are based on CPU time, not wall clock time.

⁴ Although debugging (and related corrections) may be required following a failure, in many cases the software will work properly after a restart with no other change.

Many researchers argue that MTBF is a far more useful measure than other quality-related software metrics discussed in Chapter 23. Stated simply, an end user is concerned with failures, not with the total defect count. Because each defect contained within a program does not have the same failure rate, the total defect count provides little indication of the reliability of a system. For example, consider a program that has been in operation for 3000 processor hours without failure. Many defects in this program may remain undetected for tens of thousands of hours before they are discovered. The MTBF of such obscure errors might be 30,000 or even 60,000 processor hours. Other defects, not yet discovered, might have a failure rate of 4000 or 5000 hours. Even if every one of the first category of errors (those with long MTBF) is removed, the impact on software reliability is negligible.

However, MTBF can be problematic for two reasons: (1) it projects a time span between failures but does not provide us with a projected failure rate, and (2) MTBF can be misinterpreted to mean average life span even though this is *not* what it implies.

An alternative measure of reliability is *failures in time* (FIT)—a statistical measure of how many failures a component will have over 1 billion hours of operation. Therefore, 1 FIT is equivalent to one failure in every billion hours of operation.

In addition to a reliability measure, you should also develop a measure of availability. *Software availability* is the probability that a program is operating according to requirements at a given point in time and is defined as

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$$

The MTBF reliability measure is equally sensitive to MTTF and MTTR. The availability measure is somewhat more sensitive to MTTR, an indirect measure of the maintainability of software. Of course, some aspects of availability have nothing to do with failure. For example, scheduling downtime (for support functions) causes the software to be unavailable. For a comprehensive discussion of software reliability measures, see [Laz11].

17.7.2 Use of AI to Model Reliability

Some software engineers view data science as the application of artificial intelligence techniques to solve software engineering problems. One of the things artificial intelligence methods attempt to do is provide reasonable solutions to problems where the needed data may be incomplete. *Software reliability* is defined as the probability of failure-free software operation for a specified time period in a specified environment. This means that we can never know the exact moment when a software product will fail because we will never have the complete data needed to calculate the probability.

Software engineers have used statistical techniques based on Bayes' theorem⁵ in quantitative decision-making situations for several years. *Bayesian inference* is a method of statistical inference in which Bayes' theorem is used to update the probability

⁵ Bayes' theorem for conditional probabilities is $P(A|B) = (P(B|A) * P(A)) / P(B)$. For more details, see <http://www.statisticshowto.com/bayes-theorem-problems/>.

for a hypothesis (such as system reliability) as more evidence or information becomes available. Bayesian inference can be used to estimate probabilistic quantities using historic data even when some information is missing. Using Bayesian techniques has allowed for real-time solutions to probability estimation problems that are beyond human reasoning [Tos17].

Proactive failure prediction using machine learning was discussed briefly in Section 15.4.3. It would be nice to be able to predict system failures in subsequent sprints before you deliver the prototype being developed in the current sprint. Making use of predictive data analytics such as a regression model involving MTBF has been used to estimate where and what types of defects might occur in future prototypes [Bat18].

A *genetic algorithm* is a heuristic search method used in artificial intelligence and computing. It is used for finding near-optimal solutions to search problems based on the theory of natural selection and evolutionary biology. Genetic algorithms can be used to grow reliability models by discovering relationships in historic system data. These models have been used to identify software components that may fail in the future. Sometimes these models have been created based on metrics estimated from UML models before any codes has been written [Pad17]. This type of work is very important to developers interested in refactoring software products or reusing software components in other products.

17.7.3 Software Safety

Software safety is a software quality assurance activity that focuses on the identification and assessment of potential hazards that may affect software negatively and cause an entire system to fail. If hazards can be identified early in the software process, software design features can be specified that will either eliminate or control potential hazards.

A modeling and analysis process is conducted as part of software safety. Initially, hazards are identified and categorized by criticality and risk. For example, some of the hazards associated with a computer-based cruise control for an automobile might be: (1) causes uncontrolled acceleration that cannot be stopped, (2) does not respond to depression of brake pedal (by turning off), (3) does not engage when switch is activated, and (4) slowly loses or gains speed. Once these system-level hazards are identified, analysis techniques are used to assign severity and probability of occurrence.⁶ To be effective, software must be analyzed in the context of the entire system. For example, a subtle user input error (people are system components) may be magnified by a software fault to produce control data that improperly positions a mechanical device. If and only if a set of external environmental conditions is met, the improper position of the mechanical device will cause a disastrous failure. Analysis techniques [Eri15] such as fault tree analysis, real-time logic, and Petri net models can be used to predict the chain of events that can cause hazards and the probability that each of the events will occur to create the chain.

⁶ This approach is similar to the risk analysis methods described in Chapter 26. The primary difference is the emphasis on technology issues rather than project-related topics.

Once hazards are identified and analyzed, safety-related requirements can be specified for the software. That is, the specification can contain a list of undesirable events and the desired system responses to these events. The role of software in managing undesirable events is then indicated.

Although software reliability and software safety are related to one another, it is important to understand the subtle difference between them. Software reliability uses statistical analysis to determine the likelihood that a software failure will occur. However, the occurrence of a failure does not necessarily result in a hazard or mishap. Software safety examines the ways in which failures result in conditions that can lead to a mishap. That is, failures are not considered in a vacuum but are evaluated in the context of an entire computer-based system and its environment.

A comprehensive discussion of software safety is beyond the scope of this book. If you have further interest in software safety and related system issues, see [Fir13], [Har12a], and [Lev12].

17.8 THE ISO 9000 QUALITY STANDARDS⁷

A *quality assurance system* may be defined as the organizational structure, responsibilities, procedures, processes, and resources for implementing quality management [ANS87]. Quality assurance systems are created to help organizations ensure their products and services satisfy customer expectations by meeting their specifications. These systems cover a wide variety of activities encompassing a product's entire life cycle including planning, controlling, measuring, testing, and reporting, and improving quality levels throughout the development and manufacturing process. ISO 9000 describes quality assurance elements in generic terms that can be applied to any business regardless of the products or services offered.

To become registered to one of the quality assurance system models contained in ISO 9000, a company's quality system and operations are scrutinized by third-party auditors for compliance to the standard and for effective operation. Upon successful registration, a company is issued a certificate from a registration body represented by the auditors. Semiannual surveillance audits ensure continued compliance to the standard.

The requirements delineated by ISO 9001:2015 address topics such as management responsibility, quality system, contract review, design control, document and data control, product identification and traceability, process control, inspection and testing, corrective and preventive action, control of quality records, internal quality audits, training, servicing, and statistical techniques. For a software organization to become registered to ISO 9001:2015, it must establish policies and procedures to address each of the requirements just noted (and others) and then be able to demonstrate that these policies and procedures are being followed. If you desire further information on ISO 9001:2015, see [ISO14].

⁷ This section, written by Michael Stovsky, has been adapted from *Fundamentals of ISO 9000*, a workbook developed for *Essential Software Engineering*, a video curriculum developed by R. S. Pressman & Associates, Inc. Reprinted with permission.



The ISO 9001:2015 Standard

The following outline defines the basic elements of the ISO 9001:2015 standard.

Comprehensive information on the standard can be obtained from the International Organization for Standardization (www.iso.ch) and other Internet sources (e.g., www.praxiom.com).

Establish the elements of a quality management system.

Develop, implement, and improve the system.

Define a policy that emphasizes the importance of the system.

Document the quality system.

Describe the process.

Produce an operational manual.

Develop methods for controlling (updating) documents.

Establish methods for record keeping.

Support quality control and assurance.

Promote the importance of quality among all stakeholders.

Focus on customer satisfaction.

Define a quality plan that addresses objectives, responsibilities, and authority.

Define communication mechanisms among stakeholders.

Establish review mechanisms for the quality management system.

Identify review methods and feedback mechanisms.

Define follow-up procedures.

Identify quality resources including personnel, training, and infrastructure elements.

Establish control mechanisms.

For planning.

For customer requirements.

For technical activities (e.g., analysis, design, testing).

For project monitoring and management.

Define methods for remediation.

Assess quality data and metrics.

Define approach for continuous process and quality improvement.

17.9 THE SQA PLAN

The *SQA plan* provides a road map for instituting software quality assurance. Developed by the SQA group (or by the software team if an SQA group does not exist), the plan serves as a template for SQA activities that are instituted for each software project.

A standard for SQA plans has been published by the IEEE [IEE17]. The standard recommends a structure that identifies: (1) the purpose and scope of the plan, (2) a description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA, (3) all applicable standards and practices that are applied during the software process, (4) SQA actions and tasks (including reviews and audits) and their placement throughout the software process, (5) the tools and methods that support SQA actions and tasks, (6) software configuration management (Chapter 22) procedures, (7) methods for assembling, safeguarding, and maintaining all SQA-related records, and (8) organizational roles and responsibilities relative to product quality.

17.10 SUMMARY

Software quality assurance is a software engineering umbrella activity that is applied at each step in the software process. SQA encompasses procedures for the effective application of methods and tools, oversight of quality control activities such as technical reviews and software testing, procedures for change management, procedures for assuring compliance to standards, and measurement and reporting mechanisms.

To properly conduct software quality assurance, data about the software engineering process should be collected, evaluated, and disseminated. Statistical SQA helps to improve the quality of the product and the software process itself. Software reliability models extend measurements, enabling collected defect data to be extrapolated into projected failure rates and reliability predictions.

In summary, you should note the words of Dunn and Ullman [Dun82]: “Software quality assurance is the mapping of the managerial precepts and design disciplines of quality assurance onto the applicable managerial and technological space of software engineering.” The ability to ensure quality is the measure of a mature engineering discipline. When the mapping is successfully accomplished, mature software engineering is the result.

PROBLEMS AND POINTS TO PONDER

- 17.1.** Some people say that “variation control is the heart of quality control.” Because every program that is created is different from every other program, what are the variations that we look for and how do we control them?
- 17.2.** Is it possible to assess the quality of software if the customer keeps changing what it is supposed to do?
- 17.3.** Quality and reliability are related concepts but are fundamentally different in a number of ways. Discuss the differences.
- 17.4.** Can a program be correct and still not be reliable? Explain.
- 17.5.** Can a program be correct and still not exhibit good quality? Explain.
- 17.6.** Why is there often tension between a software engineering group and an independent software quality assurance group? Is this healthy?
- 17.7.** You have been given the responsibility for improving the quality of software across your organization. What is the first thing that you should do? What’s next?
- 17.8.** Besides counting errors and defects, are there other countable characteristics of software that imply quality? What are they and can they be measured directly?
- 17.9.** The MTBF concept for reliability is open to criticism. Explain why.
- 17.10.** Consider two safety-critical systems that are controlled by computer. List at least three hazards for each that can be directly linked to software failures.